

Module 1: Lesson 1

Introduction to Machine Learning



Outline

- ▶ Machine Learning
- ▶ Time Series Momentum
- ▶ The Linear Regression model

Machine Learning (1/3): AI, ML, and DL

When we mention the concepts of Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL), we are referring to different things.

- ▶ **AI** represents the effort to automate intellectual tasks normally performed by humans. Some of these tasks involve learning; others do not.

Consider a chess program that, using nested "if" loops, determines what action to undertake under each possible location of the pieces on the board. This program does not involve learning (may not even be considered AI anymore).

- ▶ **ML** deals with computer programs that allow computers to learn from data. Humans input data as well as the answers expected from (some part) of the data, while the program dictates the rules.

A computer program is said to learn from experience with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience.

- ▶ **DL** is a strategy for learning representations from data that learns successive layers of increasingly meaningful representations. Neural networks are the most usual example of DL models.

Machine Learning (2/3): Supervised and Unsupervised learning

At the most fundamental level, ML can be categorized into two main types: Supervised Learning and Unsupervised Learning.

- ▶ **Supervised Learning** models the relationship between measured features of the data and some labels. Once the model is trained and determined, it can be applied to other data with unknown labels to make predictions.

Supervised learning is further subdivided into classification tasks and regression tasks.

- ▶ **Unsupervised Learning** involves modeling the features of a dataset without reference to any label and is often described as "letting the dataset speak for itself."

These models include tasks such as clustering and dimensionality reduction.

Machine Learning (3/3): Elements of a supervised ML model

In supervised learning, the data is represented by some labels associated with certain features.

In supervised learning, the data is represented by some labels associated with certain features.

- ▶ Features matrix (X): Composed of instances (rows) that have some input features (columns).
- ▶ Labels (also targets or output, y): A vector with the information we want the model to predict.

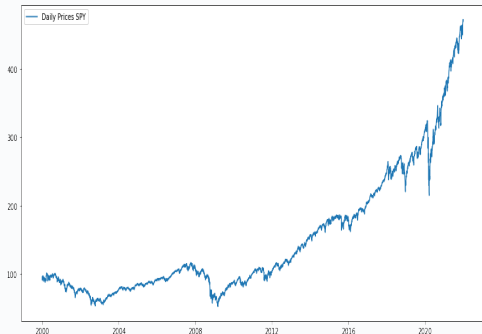
For each instance in X , there is a value, a label, in y (perhaps vector-valued).

The model is a function $g(\cdot)$ that takes the X matrix as input and tries to predict values as close as possible to the y targets.

Training: The computations that are developed to obtain the $g(\cdot)$ function.

When the model is trained, we can make predictions for the labels of data that have been unused in training (test sample).

Time Series Momentum



We are going to study the basic elements of ML outlined above in the context of a simple financial application: Time series momentum.

A "time series" momentum strategy simply buys stocks that had a positive return in the past and sells stocks that had a negative return in the past.

Our ML task in this application is to use a time series momentum strategy that exploits the predictability of SPY, an ETF that tracks the S&P500 index.

This application will illustrate the basic tools of ML and also the complicated task of developing useful predictive models of asset returns.

Linear Regression (1/4): Elements

Linear Regression imposes $g(\cdot)$ to be a linear function of the inputs to predict the labels:

$$\hat{y} = g(X) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

- ▶ $\hat{y}$ is the predicted value for the label.
- ▶ n is the number of features.
- ▶ x_j is the j^{th} input feature.
- ▶ θ_k is the k^{th} model parameter, including the bias term θ_0 .

We can train the vector θ by minimizing the mean-squared error (MSE) of the prediction in the training sample:

$$\min_{\theta} \frac{1}{m} \sum_{i=1}^m \left(y^{(i)} - x^{(i)} \theta \right)^2$$

where m is the number of instances, each represented by a label $y^{(i)}$ and a row vector of features $x^{(i)}$.

Linear Regression (2/4): Training

Minimization of the MSE leads to a closed form expression, called the *Normal Equation*:

$$\hat{\theta} = (X^T X)^{-1} X^T y \quad (1)$$

The computations involved for the inverse part are extremely costly when we have information on a large number of features or too many instances to fit in the memory of the computer.

In Machine Learning applications, we usually opt for a generic optimization algorithm that is capable of finding optimal solutions to a wide range of problems: **Gradient Descent**.

$$\theta^{[next\ step]} = \theta^{[previous\ step]} - \eta \nabla_{\theta} MSE \left(\theta^{[previous\ step]} \right)$$

where $\nabla_{\theta} MSE(\theta)$ is the gradient of the MSE with respect to θ , and η is the **learning rate**.

Linear Regression (3/4): R^2 , In-sample vs. Out-of-sample

To measure the goodness of fit of the model, we can compute the coefficient of determination, or R^2 :

$$R^2 = 1 - \frac{\sum_{i=1}^m (y^{(i)} - x^{(i)} \hat{\theta})^2}{\sum_{i=1}^m (y^{(i)} - \bar{y})^2}$$

where $\bar{y}$ is the average label in the training sample. A low R^2 in the training sample suggests **underfitting**.

The R^2 of the linear regression indicates the proportion of the labels' variance explained by the model.

- We can also understand the R^2 as the prediction gains made by the model against a prediction model that just uses the sample average to predict the labels.

In the training sample (or *in-sample*) the R^2 of the Linear Regression will always be above zero (if there is a bias term).

In the test sample (or *out-of-sample*) the R^2 of Linear Regression may be negative:

- A model may be arbitrarily bad at making predictions out-of-sample, even worse than making a prediction based on the average. A potential solution is to increase the complexity of the model.

Linear Regression (4/4): Overfitting

Machine Learning algorithms may be powerful at predicting the labels of the training sample.

However, the trained model may yield bad results when trying to generalize to new data: **Overfitting**.

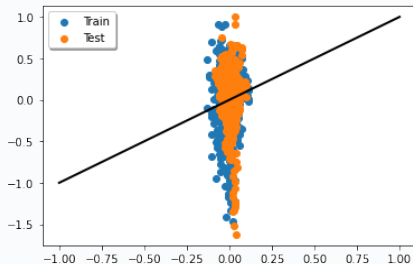
Different ways to avoid overfitting:

- ▶ Gather more training data to reduce noise or clean the data to fix errors or remove outliers.
- ▶ Regularization. Force the trainable parameters to be relatively small and avoid overfitting.
- ▶ Early stopping. Introduce a validation sample to stop training as soon as the model starts to generalize badly.
- ▶ Tuning of hyperparameters. Determine the combination that best fits the trained model in the validation sample.

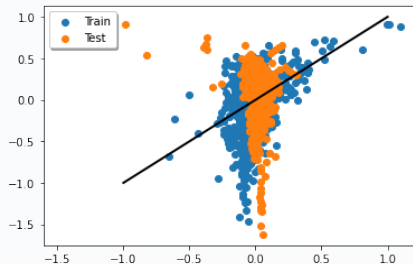
Hyperparameter: a parameter of a learning algorithm that does not explicitly serve to predict the labels. As such, hyperparameters are fixed during training.

Time Series Momentum: Results

Linear Regression:
(x-axis: Predicted return. y-axis: Actual return)



Linear Regression with powers of features:
(x-axis: Predicted return. y-axis: Actual return)



The Linear Regression model clearly underfits the data and is of little use to predict the evolution of SPY.

Increasing the complexity of the model by adding powers of the features leads to overfitting.

Summary of Lesson 1

In Lesson 1, we have looked at:

- ▶ The main distinctions between AI, ML, and DL
- ▶ The two types of Machine Learning models: Supervised and Unsupervised
- ▶ The Linear Regression model
- ▶ The overfitting problem

⇒ **References for this lesson:**

Géron, Aurélien. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 2nd ed., O'Reilly Media, Inc., 2019.

TO DO NEXT: Now, please go to the associated Jupyter Notebook for this lesson to review the concepts of Lesson 1.

In the next lesson, we will describe the main tools for parameter regularization and cross-validation.